NanoBook: Phase 1 Report

Reference order book and exchange layer · 16 October 2026 · C++23, GCC 14 / Clang 18, Linux

1. Summary

Phase 1 set out to build a slow, obviously correct order book that the phase 2 fast book can be checked against. It delivered that, plus an exchange layer, a compile-time contract for any book, and the first performance baseline.

Order types	Limit, Market, IOC, FOK, Cancel, Modify (cancel/replace), self-match prevention
Structure	One `Exchange` in front of one book per symbol, with exchange-wide order ids like Nasdaq
Tests	**109**, all passing on 4 build presets (GCC and Clang × Debug+ASan/UBSan and Release)
Bug checks	**13 deliberately planted bugs**, all caught by the tests (§7.3)
Code size	About 1,050 lines of engine code; 2,600 including tests and benchmarks
Baseline	31–137 ns per operation and **2 heap allocations per added order** (§8)
Commits	12 small commits before this report, each building and passing tests (README rule 21)

2. Background: what is being built

An **order book** is the list of every waiting buy order (bids) and sell order (asks) for one stock, sorted by price. Each price is a **level** with a first-come-first-served queue of orders.

A **matching engine** takes each incoming order and decides whether it trades. It follows **price-time priority**: the best price trades first, and within one price, the oldest order trades first. The trade happens at the waiting order's price.

```
ASKS (sellers)    $10.03  [C: 500]
                  $10.02  [A: 100] [B: 200]     <- best ask
BIDS (buyers)     $10.01  [D: 400]              <- best bid
                  $10.00  [E: 300] [F: 400]
```

A buy for 250 at $10.02 trades 100 with A, then 150 with B. B keeps its place in the queue with 50 left.

NanoBook plays the role of the core of an exchange: no accounts, money, apps or networking. Orders come from tests, random streams, and later (phase 4) a real day of Nasdaq data.

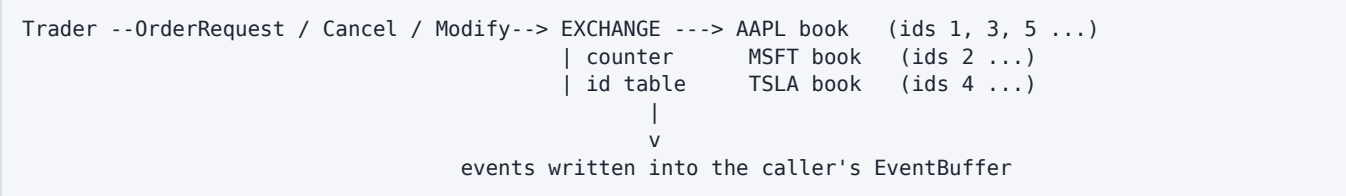
3. Phase 0 recap (the foundation)

Piece	What it gives
CMake + presets	One command per configuration: `cmake --workflow --preset gcc-debug-asan`
GCC 14 and Clang 18	Two compilers catch different mistakes
ASan + UBSan	Memory errors and undefined behaviour stop the test instead of passing silently
`-Werror` with strict warnings	Conversions, sign mistakes and shadowing are build errors
clang-format, clang-tidy	Style and suspicious patterns are checked in CI
GitHub Actions	Every push builds and tests every preset
`-fno-exceptions` (engine only)	A `throw` inside the engine does not compile (README rule 3)

4. What was built, in order

Commit	Change
Design note	`docs/design.md`: every behaviour decided **before** coding (rule 23)
Types and events	Strong types (`Price`, `Qty`, `OrderId`, `OwnerId`), inputs, the 40-byte `Event`
Limit + Cancel	The `std::map` + `std::list` reference book, invariant checker, fixed-size `EventBuffer`
Market, IOC, FOK	"Look first, then act" for FOK, so the book never has to undo a trade
Self-match prevention	A trader's new order cancels their own resting order instead of trading with it
Modify	FIX-style total-quantity rule; keep or lose queue priority
`max_qty`	Found in review: huge quantities could overflow. A fat-finger limit makes this impossible.
Exchange layer	One id counter for all symbols, an id table, exact cancel reasons
`OrderBook` concept	Compile-time contract for any book; tests run per book type
Baseline benchmarks	Time and allocations per operation, recorded in `docs/perf-log.md`

5. Architecture



Part	Job
`Exchange<Book>`	Numbers every order from one counter, routes messages, turns "not found" into the exact reason
`ReferenceBook`	One symbol. Bids and asks as `std::map<Price, std::list<Order>>`, plus `std::map<OrderId, Locator>` to find an order
`EventBuffer`	Fixed-size output, allocated once and reused for every input
`OrderBook` concept	The checklist every book must meet. The exchange only accepts types that meet it.

6. Key design decisions

Decision	Reason
Prices and quantities are integers in strong types	Floating point breaks equality. Strong types make mixing up a price and a quantity a compile error, at zero runtime cost.
FIX names for events and reasons	Interviewers recognise them. No FIX wire protocol (out of scope).
Errors are `Rejected` events, never exceptions	The engine is built with `-fno-exceptions`. A throw costs microseconds.
The order of checks is part of the contract	When several checks fail, both books must report the same one
Self-match: cancel the **resting** order	The new order reflects the strategy's current view; the old one is stale
Modify quantity = new **total**, including what already traded	A modify that arrives after a partial fill can never cause an overfill (FIX rule for 35=G)
Modify to 0 is rejected (`UseCancel`); a no-op modify is rejected (`NoChange`)	One way to remove an order; the trader is told when a message did nothing
Shrinking keeps queue priority; growing or a new price loses it	Asking for less hurts nobody behind you
`BookFull` only if the full book would have to rest an order that doesn't cross	A crossing order always frees a slot first, so it can never be wrongly rejected
`max_orders × max_qty` must fit in 64 bits	Makes signed overflow (undefined behaviour) impossible in every sum
The exchange assigns ids from one counter	Traders can't send a wrong id; reasons are exact; each book sees ITCH-like ids with gaps
Cancel and Modify carry the symbol	They route straight to the book. The id table is only read when something is wrong (cold path).
Fixed capacities, set at start-up	The fast book can allocate everything once and nothing while trading (rule 1)

7. Testing

7.1 Tests by suite

Suite	Tests	Covers
`BookTest`	70	Every order type, every design §10 row, self-match, modify, edge cases
`BookCapacityTest`	9	Full book, worst-case event count, quantity limit
`ExchangeTest`	16	One counter, routing, exact reasons (rows E1–E9)
`ExchangeCapacityTest`	1	Running out of ids
`ExchangeConfigTest`	2	Config validity, buffer size
`Types`, `BookConfig`, `Event`	11	Strong types, config rules, event equality
Total	**109**	

The book and exchange tests are **typed tests**. Adding the phase 2 fast book to one list (`BookTypes`) runs all 96 of them against it.

7.2 Invariants, checked after every single input

1. The book is not crossed (best bid below best ask).

2. No empty price levels, and no resting order with quantity ≤ 0.
3. Each level's quantity equals the sum of its orders.
4. Every order in the id lookup is in exactly one level, on the right side and price.
5. Within a level, orders are in arrival order.
6. Filled + remaining = total, for every order.
7. Exchange: every resting order's id-table entry points to its own book.

7.3 Planted bugs (do the tests have teeth?)

Each bug was put in on purpose, the tests were run, and the code was restored.

Area	Planted bug	Result
Matching	Empty price level left in the book	First a **hang**; after adding an `assert`, an instant crash naming the line
Matching	Newest order matched first (breaks FIFO)	1 test failed
FOK	Look-first check skipped	`assert` fired; wrong events in Release
IOC	Remainder rests instead of being cancelled	5 tests failed
Self-match	Matching ignores the owner	8 tests failed
Self-match	Look-first count includes own orders	Market test failed, FOK `assert` fired
Modify	Growing at the same price keeps priority	2 tests failed
Modify	Quantity treated as "remaining" (overfill)	1 test at first; **gap found**, a test added, then 2
Limits	`max_qty` check removed	2 tests failed
Exchange	No cold-path refinement	4 tests failed
Exchange	Refinement ignores which book	1 test failed
Exchange	Rejected orders written to the id table	1 test failed
Concept	A book missing `Modify`	Compile error naming `book.handle(modify, out)`

7.4 Problems found and fixed during phase 1

Problem	How it was found	Fix
Signed overflow with huge quantities	UBSan, in a review test	`max_qty` plus a config check
A bug could hang CI for 25 minutes	Planted bug	60-second test timeout, plus an `assert` that fails fast
Overfill rule had a single test, on one code path	Planted bug	Added `ModifyUpCountsWhatAlreadyTraded`
`OrderRequest` was 32 bytes, not 24	Its own `static_assert`	Reordered fields to remove padding
Design gaps (check order, exact `BookFull` rule)	Writing the code	Added to the design note

8. Baseline performance

Measured on an Intel Core i7-14650HX, WSL2, GCC 14 `-O3`, pinned to one core, median of 5 runs (variation below 1%). **For comparison only, never for publishing** (rule 20). Full environment in `docs/perf-log.md`.

Operation	Time per op	Allocations per op
Add a resting limit order	58.9 ns	**2.01**
Cancel a resting order	136.9 ns	0
Order that trades fully with one resting order	31.5 ns	0
Sweep 1 / 10 / 100 price levels	40 ns / 386 ns / 3.98 µs	0
Mixed stream (60% add, 30% cancel, 10% IOC)	83.6 ns	**1.26**

8.1 Why cancel is the slowest operation

A cancel does **two map lookups**, then unlinks a list node:

1. `orders_.find(id)`: the id lookup, to find where the order is.
2. `levels.find(price)`: the price-level map, to find its level so the order can be removed and an empty level erased.
3. Erase the order's `std::list` node.

`std::map` is a **red-black tree**. Every node is a separate heap allocation, placed wherever the allocator has room, so neighbouring keys are **not** next to each other in memory. A lookup starts at the root and follows a pointer down one level at a time: about 14 hops for 10,000 orders, and about 7 for 100 price levels. Each hop can land on memory the CPU hasn't loaded, which is a **cache miss** (tens of ns from L3, about 100 ns from main memory). The two maps are separate trees in separate memory, and the list node is a third place. The benchmark also cancels in shuffled order, so the CPU can't predict what to load next.

Contrast that with an array: elements sit side by side, the CPU loads them 64 bytes (one cache line) at a time, and it prefetches the next ones when access is sequential. That is why phase 2 replaces the trees with pools, indices and flat arrays.

One more observation: the second lookup isn't strictly necessary even here. `std::map` iterators stay valid, so `Locator` could store the level's iterator and skip lookup 2. The reference book keeps the simpler form on purpose, because clarity is its job. The fast book won't need either lookup.

8.2 Why these numbers flatter the `std::map` book

Every benchmark starts from a fresh heap, so new nodes sit close together and stay in cache. After a real trading day the nodes are scattered, and lookups miss cache far more often. Phase 3 (tail latency) and phase 4 (a full-day replay) will show this.

---

9. Lessons worth keeping

Lesson	Example from phase 1
Fix the cause, make the symptom loud	An empty level is removed by code; an `assert` catches it if that code ever breaks. Never "self-heal".
Tests only catch what they set up	FIFO bugs only show with several orders at one price
Plant bugs to test the tests	It found the single-test overfill rule
`static_assert` the size of hot structs	It caught 8 bytes of padding
Don't silence a warning; change the code	A GCC false positive with `std::optional` was avoided with `std::unique_ptr`
Templates hide base-class members	Typed tests need `this->h`
Design first	Three contract gaps surfaced while coding and went back into the note

---

10. Known limitations

- Debug-only `assert` checks vanish in Release builds, so an undersized `EventBuffer` or invalid config isn't checked there. By design, the caller is responsible and Debug tests catch both.
- The id table costs 2 bytes per order of the session, fixed at start-up (about 200 MB for 100M orders).
- Benchmarks report averages from WSL2. Tail latency and publishable numbers come in phase 3.
- Moving price bands, auctions, halts, threads and networking are out of scope.

---

11. Next: phase 2, the fast book

1. **Design note first** (rule 23): order pool, intrusive lists, id lookup, price-level structure.
2. **Zero allocations after start-up**, proven by the allocation counter.
3. **Same behaviour**: the 96 typed tests, plus a differential fuzzer comparing event streams with the reference book over 10 million or more random operations.
4. **Faster**, shown against this baseline in `docs/perf-log.md`, with a before/after table for every change (rule 22).